

Software Requirement Specification (SRS)

Software Component Cataloguing System

Prepared by: Shubham Singh

March 30, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Intended Audience	3
1.4	Definitions	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Classes	5
2.4	Tech Stack	5
3	System Features	7
3.1	Feature 1: Add Component	7
3.2	Feature 2: Search Component	7
3.3	Feature 3: Usage Tracking	7
3.4	Feature 4: User Registration (Sign Up)	8
3.5	Feature 5: User Login	8
4	Data Design	9
4.1	Database Schema	9
4.1.1	User Table	9
4.1.2	Component Table	9
4.1.3	Catalogue Table	9
4.1.4	Entity Relationship Overview	10
5	System Models	11
5.1	Use Case Diagram	11
5.2	Data Flow Diagram	11
5.3	Architecture Diagram	11
6	Non-Functional Requirements	12
6.1	Performance	12
6.2	Security	12
6.3	Reliability	12
6.4	Scalability	12
7	Charts and Analysis	13
7.1	Component Usage Distribution	13
8	Conclusion	14

1 Introduction

1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to provide a detailed description of the **Software Component Cataloguing System**. This system is designed to store, manage, and retrieve reusable software components efficiently.

In modern software development, reuse of components significantly reduces development time and cost. However, without proper organization, locating reusable components becomes difficult. This system addresses that issue by providing a centralized catalogue.

1.2 Scope

The system will:

- Maintain a repository of reusable software components
- Support both design (UML, ERD) and code components
- Allow keyword-based searching
- Track usage statistics of components
- Support hierarchical classification of components

1.3 Intended Audience

This document is intended for:

- Developers
- System Designers
- Project Managers
- Testers

1.4 Definitions

Term	Description
Component	A reusable piece of software (code or design)
Catalogue	Database storing components
Keyword	Tag used for searching components
Metadata	Information describing components

2 Overall Description

2.1 Product Perspective

The Software Component Cataloguing System is a standalone application that acts as a repository for reusable components. It can also be integrated into larger development environments.

2.2 Product Functions

The major functions of the system include:

- Adding new components
- Deleting existing components
- Updating component information
- Searching components using keywords
- Viewing usage statistics

2.3 User Classes

- **User (Developer):**
 - The user interacts with the system to search, browse, and utilize reusable software components.
 - Users can query the catalogue using keywords to find relevant components.
 - Users can view component details such as type, usage statistics, and associated metadata.
 - Users indirectly contribute to the system by increasing usage statistics when components are accessed or reused.

2.4 Tech Stack

The Software Component Cataloguing System is developed using a modern web-based technology stack to ensure scalability, performance, and ease of maintenance.

- **Frontend:**
 - React.js for building an interactive and dynamic user interface
 - JavaSwing for structure and styling

- **Backend:**
 - Spring boot for handling server-side logic and API development
 - RESTful APIs for communication between frontend and backend
- **Database:**
 - SQL (mySQL database) for storing component data and metadata
 - Flexible schema to handle different types of components (code/design)
- **Operating System Support:**
 - Compatible with Windows, Linux, and MacOS environments

3 System Features

3.1 Feature 1: Add Component

Description: The user can add new software components into the catalogue. This feature allows the system to grow continuously by storing reusable components along with their associated metadata.

Inputs:

- Component Name
- Type (Code/Design)
- Keywords

Processing:

- The system validates the input data provided by the user
- Ensures that mandatory fields are not empty
- Stores the component information in the database

Output:

- Confirmation message indicating successful addition of the component

3.2 Feature 2: Search Component

Users can search components using keywords.

- Supports multiple keyword search
- Returns ranked results

3.3 Feature 3: Usage Tracking

System maintains:

- Number of times component used
- Number of times searched but not used

3.4 Feature 4: User Registration (Sign Up)

Description: This feature allows a new user to create an account in the system. Registration is required to access the functionalities of the Software Component Cataloguing System.

Inputs:

- Username
- Email Address
- Password
- Confirm Password

Processing:

- The system validates all input fields
- Checks whether the email already exists in the database
- Verifies password strength and confirmation match
- Encrypts the password for secure storage
- Stores the user details in the database

Output:

- Successful registration message
- Redirect to login page

3.5 Feature 5: User Login

Description: This feature allows registered users to securely log in to the system and access its functionalities such as searching and managing components.

Inputs:

- Email Address
- Password

Processing:

- The system verifies the entered credentials with stored data
- Password is validated using encrypted comparison
- If credentials are correct, user session is created

Output:

- Successful login and access to dashboard
- Error message for invalid credentials

4 Data Design

4.1 Database Schema

The system database is designed to efficiently manage users, components, and catalogues. The schema follows a relational structure where entities are interconnected to support component organization and retrieval.

4.1.1 User Table

Field	Type	Description
UserID	Integer	Unique identifier for each user
Email	String	User's email address (unique)
Password	String	Encrypted user password
Username	String	Display name of the user

Relationship: A user can create and manage multiple catalogues.

4.1.2 Component Table

Field	Type	Description
ComponentID	Integer	Unique identifier for each component
ComponentName	String	Name of the component
Description	Text	Detailed explanation of the component
Keywords	String	Tags used for searching

Relationship: A component can belong to one or more catalogues.

4.1.3 Catalogue Table

Field	Type	Description
CatalogueID	Integer	Unique identifier for catalogue
CatalogueName	String	Name of the catalogue
Description	Text	Description of the catalogue
Keywords	String	Tags for searching catalogues

Relationship:

- A catalogue contains multiple components

- A user can own multiple catalogues

4.1.4 Entity Relationship Overview

- One User $\rightarrow$ Many Catalogues
- One Catalogue $\rightarrow$ Many Components
- Components can be reused across multiple catalogues

This relational design ensures efficient storage, retrieval, and reuse of software components within the system.

5 System Models

5.1 Use Case Diagram

Admin → Add/Delete Component User → Search Component

Figure 5.1: Use Case Diagram

5.2 Data Flow Diagram

User → Query → System → Database → Result

Figure 5.2: DFD

5.3 Architecture Diagram

Frontend → Backend → Database

Figure 5.3: System Architecture

6 Non-Functional Requirements

6.1 Performance

- Response time $\leq$ 2 seconds
- Efficient search algorithm

6.2 Security

- Authentication required
- Role-based access control

6.3 Reliability

- 99% uptime

6.4 Scalability

System should handle thousands of components efficiently.

7 Charts and Analysis

7.1 Component Usage Distribution

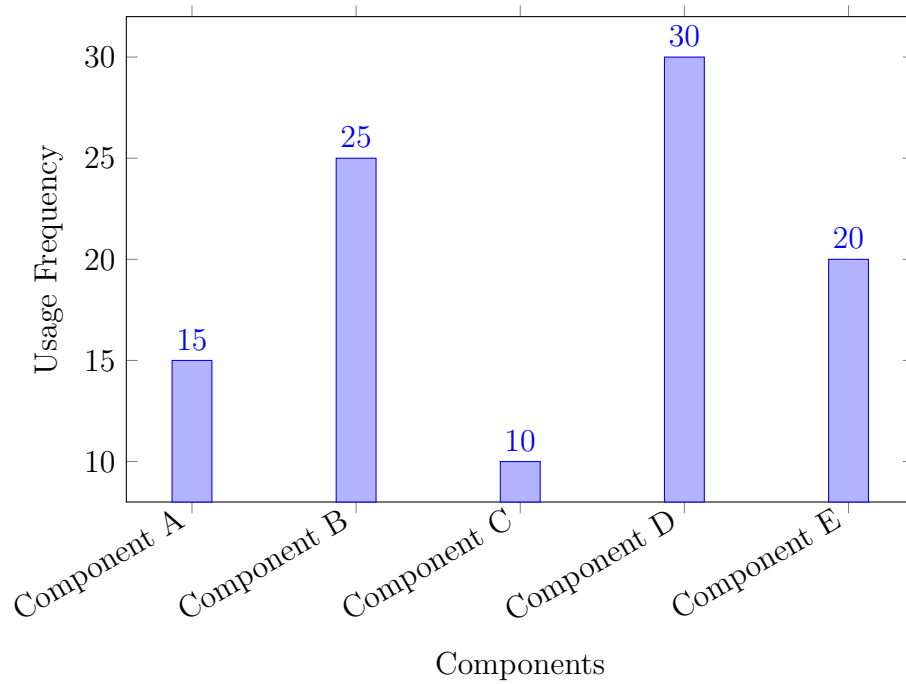


Figure 7.1: Bar Chart showing usage frequency of components

8 Conclusion

The Software Component Cataloguing System enhances software development by promoting reuse, reducing redundancy, and improving productivity. It provides an efficient mechanism for storing and retrieving reusable components.